EC-231 Operating Systems

BS (CE)-2k21

Project Proposal
Implementation of Page Replacement Algorithm

Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz
21-CE-021	Yawar Ali Malik
21-CE-020	Muhammad Ibthesam

Department of Computer Engineering

HITEC University Taxila

Engr.

Fasih Ahmed

Page Replacement Algorithm

Contents

Page Replacement Algorithm

Abstract:

Page replacement algorithms are critical for managing a computer's memory by deciding which pages to swap out when new pages need to be loaded. They aim to optimize performance by minimizing page faults. Common algorithms include FIFO (First-In-First-Out), LRU (Least Recently Used), and Optimal Page Replacement, each with distinct strategies for determining which page to replace. These algorithms balance efficiency and complexity to maintain effective memory utilization. Research in this area continues to evolve, enhancing system performance and resource management.

Introduction:

Paging is a type of memory management scheme which the computer uses to store and receive data from the secondary storage and uses that in the main memory. This scheme allows the physical address space of a process to be non-contiguous.

E.g.

There is a process of size: 4

Bytes The size of a single page: 2

Bytes Number of pages/Process: 4/2 = 2

To follow the concept of paging the MMU makes a page table. Every process has its own page table. The number of pages in a page table is the same as the number of pages in a process.

E.g. The process 1 above, will have two entries in its page table. The page table however contains the “Frame No” which is the address in main memory where the data required is stored.

Problem Statement:

Developing a paging memory management system is essential for efficient data handling in computers. This project aims to design an optimized paging mechanism where the Memory Management Unit (MMU) maintains page tables for each process, correlating virtual addresses to physical memory locations. Challenges include optimizing page table size, minimizing access latency, and ensuring smooth data transfer between secondary and primary storage, providing an efficient solution for modern computing.

Objectives:

The objective of this project is to implement a paging memory management system to efficiently manage data in computer systems. This involves designing an optimized paging mechanism where the Memory Management Unit (MMU) maintains page tables for each process. The goal is to address challenges such as optimizing page table size, minimizing access latency, and ensuring seamless data transfer between secondary and primary storage, ultimately enhancing overall system performance and resource utilization.

Expected Outcome:

The expected outcome of implementing an effective page replacement algorithm is a reduction in the number of page faults, leading to improved system performance and faster execution times. Enhanced memory utilization ensures that the most frequently accessed data is readily available, minimizing delays. This contributes to an overall smoother and more efficient operation of applications. Additionally, a well-chosen algorithm can extend the lifespan of hardware by reducing unnecessary wear from excessive data swapping.

Components used:

The Components used of page replacement algorithm following are:

1.Ubnutu

2.Sublime Text editior

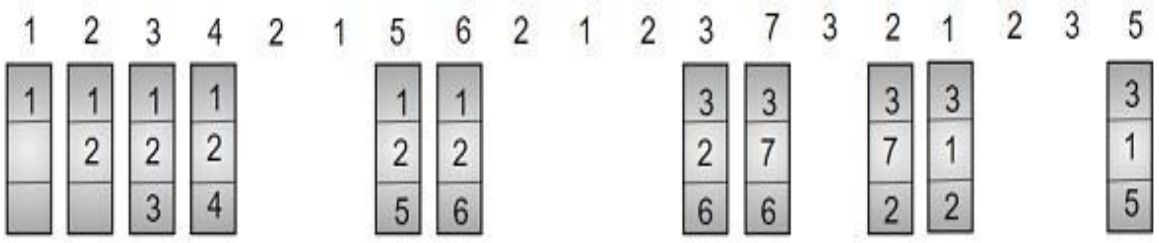
3.GCC Compiler

4.Terminal

Methodology:

The methodology for evaluating page replacement algorithms involves simulating different algorithms under various workloads to measure their performance. Key steps include defining the criteria for page replacement, such as recency of use or arrival order. Implementing the algorithms in a controlled environment allows for the collection of metrics like page fault rates and execution times. Comparing these metrics across algorithms helps identify the most efficient approach. Finally, analyzing the results provides insights into which algorithm best balances performance and complexity for the given application.

Diagram:



Conclusion:

In conclusion, the developed paging memory management system optimizes data handling in computer systems. By maintaining page tables for each process, minimizing access latency, and ensuring efficient data transfer, overall system performance is enhanced. Further refinement will continue to advance modern computing technologies.